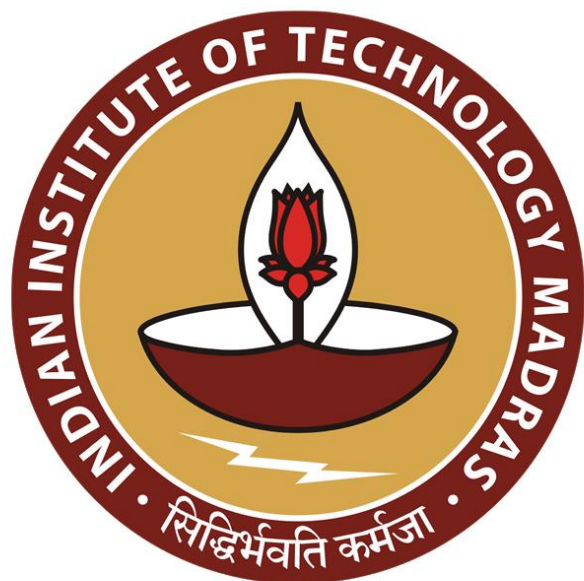




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Controlled Interaction with Objects

(Refer Slide Time: 00:13)



Controlled interaction with objects

Madhavan Mukund
<https://www.cmi.ac.in/~madhavan>

Programming Concepts using Java
Week 4



So, in the last lecture we talked about private classes and hinted that private classes can be used along with interfaces for controlled interaction with objects. So, let us explore that theme.

(Refer Slide Time: 00:23)



Manipulating objects

- Encapsulation is a key principle of object oriented programming
 - Internal data is private
 - Access to the data is regulated through public methods
 - Accessor and mutator methods

```
public class Date {  
    private int day, month, year;  
  
    public void getDay(int d) {...}  
    public void getMonth(int m) {...}  
    public void getYear(int y) {...}  
  
    public void setDay(int d) {...}  
    public void setMonth(int m) {...}  
    public void setYear(int y) {...}  
}
```



Madhavan Mukund | Controlled interaction with objects | Programming Concepts using Java | 2/6

Manipulating objects

- Encapsulation is a key principle of object oriented programming
 - Internal data is private
 - Access to the data is regulated through public methods
 - Accessor and mutator methods
- Can ensure data integrity by regulating access

```
public class Date {
    private int day, month, year;

    public void getDay(int d) {...}
    public void getMonth(int m) {...}
    public void getYear(int y) {...}

    public void setDay(int d) {...}
    public void setMonth(int m) {...}
    public void setYear(int y) {...}
}
```



Madhavan Mukund

Controlled interaction with objects

Programming Concepts using Java 2/6

Manipulating objects

- Encapsulation is a key principle of object oriented programming
 - Internal data is private
 - Access to the data is regulated through public methods
 - Accessor and mutator methods
- Can ensure data integrity by regulating access
- Update date as a whole, rather than individual components

```
public class Date {
    private int day, month, year;

    public void getDay(int d) {...}
    public void getMonth(int m) {...}
    public void getYear(int y) {...}

    public void setDate(int d, int m, int y) {
        ...
        // Validate d-m-y combination
    }
}
```



Madhavan Mukund

Controlled interaction with objects

Programming Concepts using Java 2/6

सिद्धिर्भवति कर्मजा

- Encapsulation is a key principle of object oriented programming
 - Internal data is private
 - Access to the data is regulated through public methods
 - Accessor and mutator methods
- Can ensure data integrity by regulating access
- Update date as a whole, rather than individual components
- Does this provide sufficient control?

```
public class Date {
    private int day, month, year;

    public void getDay(int d) {...}
    public void getMonth(int m) {...}
    public void getYear(int y) {...}

    public void setDate(int d, int m, int y) {
        ...
        // Validate d-m-y combination
    }
}
```



Madhavan Mahendran

Controlled interaction with objects

Programming Concepts using Java 2/6

So, encapsulation, as we know, is a key principle of object-oriented programming. So, the whole point of object-oriented programming is to ensure that the implementation details of an abstract data type of private, and you access this data through public method so that you can regulate how this access happens. So, typically, we have these two types of public methods, we have the accessor methods and we have mutator methods.

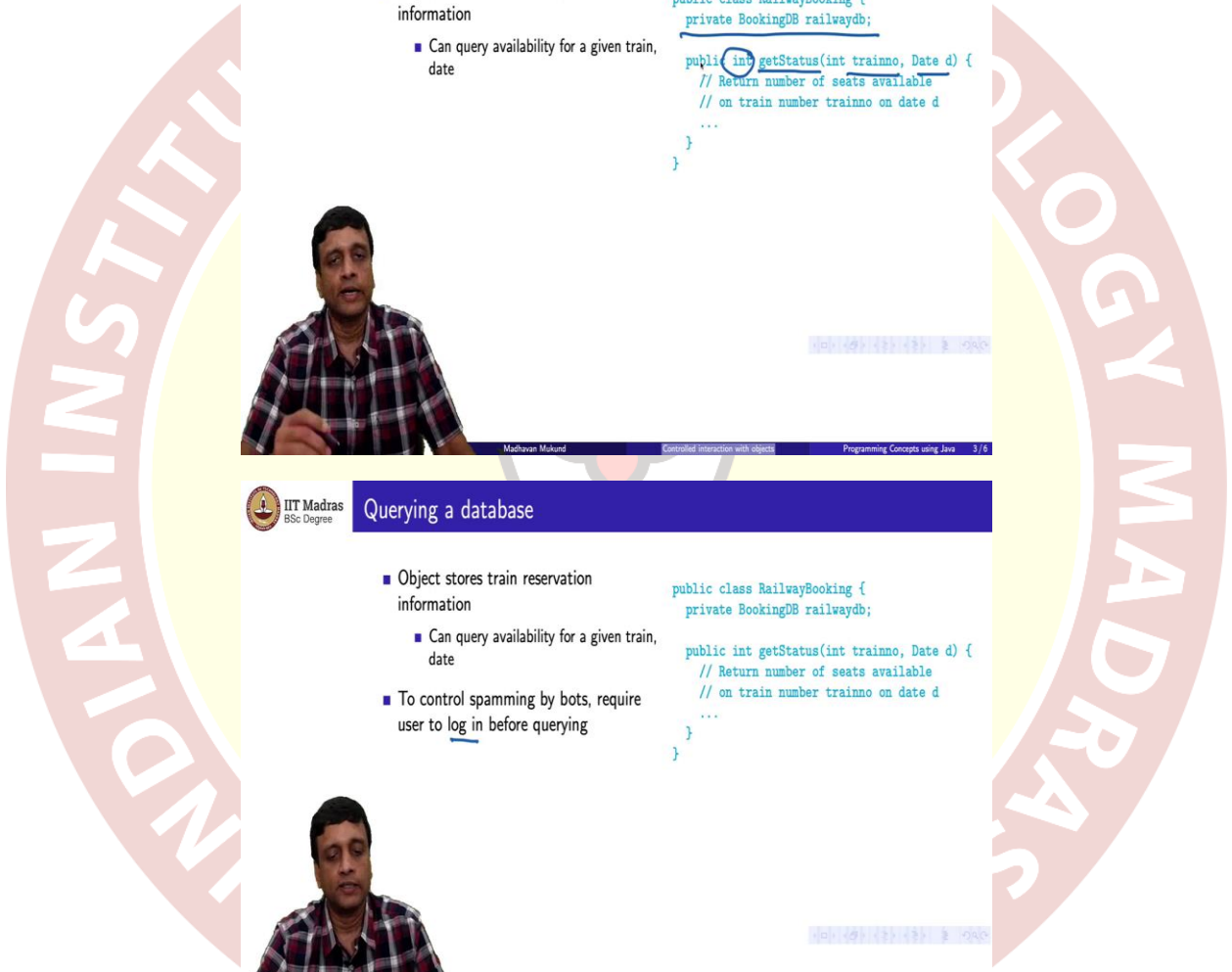
So, here is a typical example that we have seen before. So, you have a date, and it has a day, month and year. And then you have these three accessor methods which allow us to get each of these instance variables back from the current object. And we have a way to set each of these instance variables back. So, this is typically how encapsulation works. You make the data private and you have public functions, which allow you to read and write that value.

Now, we saw that this is not necessarily a good thing, because if we are allowed to set the day, month and year separately, we can put incompatible dates, we can put, for example, the 31st of February. So, we said that we would like to ensure even data integrity so we can actually regulate this access even further.

We are not obliged to write these mutator methods for each instance variable individually, instead we can create a cumulative set date function, which would set simultaneously the day, month and the year and then inside this function we can validate that this combination of day, month and year are actually sensible, and not allow this date to get into an invalid state. So, this is a simple way in which we have seen regulated access to an object by just controlling the

structure of the accessor and mutator methods. So, the question we asked him today is, can we demand more control, is there a justification for having even more control than this on how you access an object.

(Refer Slide Time: 02:20)



 **Querying a database**

- Object stores train reservation information
 - Can query availability for a given train, date

```
public class RailwayBooking {  
    private BookingDB railwaydb;  
    public int getStatus(int trainno, Date d) {  
        // Return number of seats available  
        // on train number trainno on date d  
        ...  
    }  
}
```

 Madhavan Mukund | Controlled interaction with objects | Programming Concepts using Java | 3/6

 **Querying a database**

- Object stores train reservation information
 - Can query availability for a given train, date
 - To control spamming by bots, require user to log in before querying

```
public class RailwayBooking {  
    private BookingDB railwaydb;  
    public int getStatus(int trainno, Date d) {  
        // Return number of seats available  
        // on train number trainno on date d  
        ...  
    }  
}
```

 Madhavan Mukund | Controlled interaction with objects | Programming Concepts using Java | 3/6

Querying a database

- Object stores train reservation information
 - Can query availability for a given train, date
- To control spamming by bots, require user to log in before querying
- Need to connect the query to the logged in status of the user

```
public class RailwayBooking {
    private BookingDB railwaydb;

    public int getStatus(int trainno, Date d) {
        // Return number of seats available
        // on train number trainno on date d
        ...
    }
}
```

Access



Madhavan Mukund

Controlled interaction with objects

Programming Concepts using Java 3/6

Querying a database

- Object stores train reservation information
 - Can query availability for a given train, date
- To control spamming by bots, require user to log in before querying
- Need to connect the query to the logged in status of the user
- "Interaction with state"

```
public class RailwayBooking {
    private BookingDB railwaydb;

    public int getStatus(int trainno, Date d) {
        // Return number of seats available
        // on train number trainno on date d
        ...
    }
}
```



Madhavan Mukund

Controlled interaction with objects

Programming Concepts using Java 3/6

So, as a running example, let us imagine that we are trying to query a database such as a railway reservation system. So, we have abstractly an object which stores reservation data about all the trains which are running in a railway network. And one of the things that we can access, of course, the database is a private thing, so let us assume that we have a kind of booking database class which stores this concrete railway DB as an instance. So, railway DB is the populated database of current bookings.

So, one of the accessor methods, so we are not even talking about mutator methods, so in the context of the date, the access control that we said was that we wanted to regulate how the date was updated. But now here we are talking about how to get information from the database. So,

supposing we have a function, which allows us to get the status for a given train. So, we provide a train number and we provide a date and we would like to know how many seats are available, so you get back an integer. So, this integer is 0. It means that the train is full. The integer is positive, it means that there are seats available. So, it returns the number of seats available on a particular train given by the train number we specify on a particular date.

Now, the problem with all such systems anywhere is that they are prone to attacks of different kinds. So, one kind of attack might just be a denial of service, people call it just to block it and other thing is that you might have some unscrupulous operators who are trying to get a lot of information for other purposes. They are not booking for themselves, but they are trying to get an overall picture and maybe try to optimize or somehow black-market tickets.

So, if you want to control this kind of spamming, so typically people write these bots. They write code which actually goes and keeps asking this question and you keep incrementing the train number and the date and you can get all the trains and all the days in a given window, and then you can start optimizing some way in which you will go and book tickets and then make scarcity or whatever it is that you want to do. So, to prevent some kind of unethical behavior, you might want to control this.

So, one of the things that you might want to do is you might want the user to log in before they make a query. So, you do not support this query directly. You expect a user to first register themselves with the database and only then are you allowed to ask for the status. So, you need to connect each query made through this get status to the logged in status of a user.

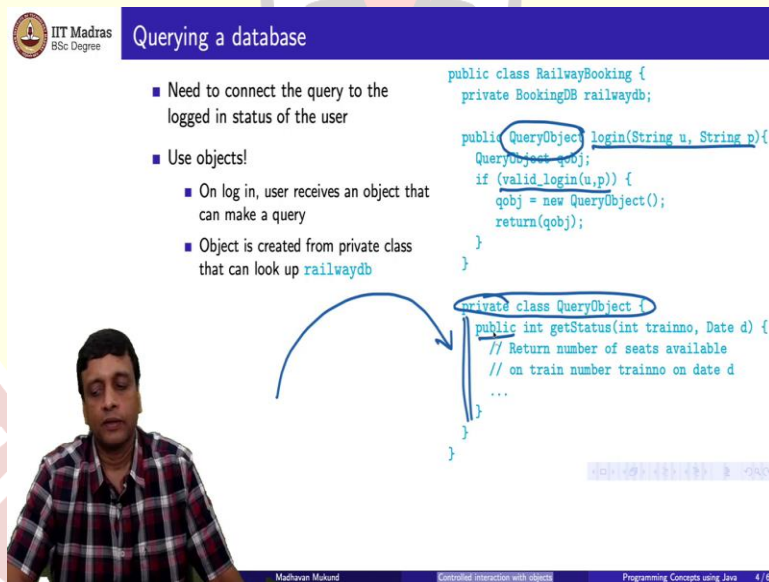
So, now the question really is how would you do this? So, you can store the status of the object, but the status of the object is different from the status of the person using the object. So, of course, one could create a list or an array or some collection of users who are currently logged in. But still when somebody actually makes a query, how do you validate that the person who makes the query is actually logged in.

Of course, you could make it very tedious and ask that person to provide here also the kind of user, day, name. So, you cannot just make an anonymous query, you cannot just say, give me the situation for this train number and this date, you have to say, give me the train number and this date, and I am this user, and then you can validate whether that user is currently logged in or not.

But obviously this now becomes a kind of uncomfortable way of doing it, because you may put this clumsy extra parameter which has nothing to do with get status, you want to regulate access to get status, but get status should not change the signature. I mean, it does not make any sense for get status to require the user to be part of the signature.

So, how are we going to get this kind of regulated access? So, the, to summarize we want the user to log in, and only if the user is logged in can they make a query. So, we have to somehow combine these two facts that the user is logged in and then the query is valid. So, this kind of an interaction is sometimes called an interaction with state. We will come back to this a little later. But basically, the interaction whether or not get status is allowed depends on the state of the user or the state of the query. So, is it a logged in get status or a not logged in get status. So, it is not just a query, it is a query which comes with some extra information that determines whether or not this query is going to be answered in the way that you would expect.

(Refer Slide Time: 06:39)



Querying a database

- Need to connect the query to the logged in status of the user
- Use objects!
 - On log in, user receives an object that can make a query
 - Object is created from private class that can look up `railwaydb`

```
public class RailwayBooking {
    private BookingDB railwaydb;

    public QueryObject login(String u, String p){
        QueryObject qobj;
        if (valid_login(u,p)) {
            qobj = new QueryObject();
            return(qobj);
        }
    }

    private class QueryObject {
        public int getStatus(int trainno, Date d) {
            // Return number of seats available
            // on train number trainno on date d
            ...
        }
    }
}
```

Madhavan Mukund | Controlled interaction with objects | Programming Concepts using Java | 4/6

Querying a database

- Need to connect the query to the logged in status of the user
- Use objects!
 - On log in, user receives an object that can make a query
 - Object is created from private class that can look up `railwaydb`
- How does user know the capabilities of private class `QueryObject`?

```
public class RailwayBooking {
    private BookingDB railwaydb;

    public QueryObject login(String u, String p){
        QueryObject qobj;
        if (valid_login(u,p)) {
            qobj = new QueryObject();
            return(qobj);
        }
    }

    private class QueryObject {
        public int getStatus(int trainno, Date d) {
            // Return number of seats available
            // on train number trainno on date d
            ...
        }
    }
}
```



Madhavan Mahand

Controlled interaction with objects

Programming Concepts using Java 4/6

Querying a database

- Need to connect the query to the logged in status of the user
- Use objects!
 - On log in, user receives an object that can make a query
 - Object is created from private class that can look up `railwaydb`
- How does user know the capabilities of private class `QueryObject`?
- Use an interface!
 - Interface describes the capability of the object returned on login

```
public interface QIF {
    public abstract int
    getStatus(int trainno, Date d);
}

public class RailwayBooking {
    private BookingDB railwaydb;
    public QIF login(String u, String p){
        QueryObject qobj;
        if (valid_login(u,p)) {
            qobj = new QueryObject();
            return(qobj);
        }
    }

    private class QueryObject implements QIF {
        public int getStatus(int trainno, Date d){
            ...
        }
    }
}
```



Madhavan Mahand

Controlled interaction with objects

Programming Concepts using Java 4/6

So, the question is how to connect the query that we want to make to the logged in status of the user. And as you would expect, the way to maintain any state information is to create an object. So, what we will do is that, when we log in, as before, by giving the user name and the password, the first thing we do is we will check that this user name and password is valid using whatever criterion, we have a database of usernames and passwords. So, there is some abstract way of validating with it.

And now if this is the case, then we will return an object of type query object. So, the user does not directly query, the user gets back something through which he or she can query. So, the user gets back a query object and query object contains inside it this gets status. So, the point about

query object is that it is a private class. So, the user does not really have access to this query object directly. You cannot, the outside user cannot create query objects, because the query object is a private class defined inside railway booking.

So, railway booking can create objects of this class. But once you return a query object then from outside the user has now access to this class which was created inside. So, it was a private or privately created object, which was passed out access to it or a reference it was given out. And now because this function is their public, you can now ask for the status from outside. So, this is the general principle that we are going to use.

But there is still a problem with this. The problem is that this class definition is private. So, how does the user of the Java compiler or whatever know that this function gets status is allowed in this object? So, how do you match up this capability of this particular function with this object which is coming from a private class which is hidden from outside? So, as you would expect, this is what we had mentioned earlier, you have to combine this notion of a private class with the idea of an interface.

So, what we have to say is that this particular class is private, but what is known about it is that it implements a public interface. So, we have a public interface, a query interface, which has this abstract function get status. And now what returns from railway booking is an object of type QIF. So, remember that when you have abstract classes or interfaces, you cannot create objects like that, but you can assign types like that. So, it gets back an object of type QIF, but the actual object it gets is going to be one which is created query object. But query object itself is internally known to implement QIF, so this, all these types now match up.

So, you create a private query object. This private query object implements this public interface QIF. Public interface QIF has this get status so the outside user knows that this object that is coming back from this railway booking can be queried using get status. So, all the pieces now fall together. You have something which is under the control of the railway booking. It is not, nobody from outside can create this query object. This query object gets created only when the user logs in. So, when the person has access to this object, you know the person has logged in. So, therefore we have been able to connect the query to the logged in status of the user.

(Refer Slide Time: 10:23)



Querying a database

- Query object allows unlimited number of queries

```
public interface QIF{
    public abstract int
        getStatus(int trainno, Date d);
}

public class RailwayBooking {
    private BookingDB railwaydb;
    public QIF login(String u, String p){
        QueryObject qobj;
        if (valid_login(u,p)) {
            qobj = new QueryObject();
            return(qobj);
        }
    }
    private class QueryObject implements QIF {
        public int getStatus(int trainno, Date d){
            ...
        }
    }
}
```



Madhavan Mukund

Controlled interaction with objects

Programming Concepts using Java 5/6



Querying a database

- Query object allows unlimited number of queries
- Limit the number of queries per login?
- Maintain a counter
 - Add instance variables to object returned on login
 - Query object can remember the state of the interaction

```
public class RailwayBooking {
    private BookingDB railwaydb;
    public QIF login(String u, String p){
        QueryObject qobj;
        if (valid_login(u,p)) {
            qobj = new QueryObject();
            return(qobj);
        }
    }
    private class QueryObject implements QIF {
        private int numqueries;
        private static int QLIM;
        public int getStatus(int trainno, Date d){
            if (numqueries < QLIM){
                // respond, increment numqueries
            }
        }
    }
}
```



Madhavan Mukund

Controlled interaction with objects

Programming Concepts using Java 5/6

Now, we can go even further than this. So, as such now this query object that we have created is created at the request of one user and this user can use this as many times as possible. But of course, there is nothing now to prevent this user from passing that query object to another part of the program to another user, for example, who has not logged in. So, then you could have an unlimited number of people making an unlimited number of queries based on a single login. So, you might now be very bureaucratic and say that each query object has a finite life. It can only allow a finite number of queries.

So, how do you maintain this? Well, this is an object. So, you can put whatever you want into it. So, for example, if we want to limit the number of queries per login, you can maintain a counter. So, inside this query object, you can maintain a counter, which is called num queries, which will be set to 0 when the object is created. And you have internally a kind of constant, so this maybe should not be static, but final, but it does not matter. So, each object has a query limit which is defined. And now you take this query limit and you check whether the number of queries made so far exceeds this query limit or not.

If it does not exceed the number of query limits and you respond to the query, but you increment. So, over time the number of queries that you can make on an object will keep incrementing this internal counter. And once its internal counter reaches the limit then it will stop responding. So, this guarantees that this counter cannot be misused by somebody else.

So, the state of the interaction in this case is both the logged in status of the user, which you can only indirectly validate that somebody got the object because they logged in, but you cannot promise it did not go to someone else. But separately the state also maintains how many times this query has been asked. And therefore, you can control the number of times. So, this is why we call this interaction with state. So, there is an interaction. And accompanying that interaction, there is a state, how many times this has happened, and so on.

We could also have separately we saw earlier examples of time, and so on, so you can keep track of time. So, you could say that this object has a lifetime of 15 minutes or half an hour. This is what happens, for instance, when you log into a bank or something. If you log into a bank and you do not press any key, then the session will terminate after certain amount of time. So, there could be a lifetime in terms of calendar time or there could be a lifetime in terms of usage, like we have done here in terms of number of invocations of a query.

But all these things can be tracked, kept track of this private object which is controlling the interaction. So, the whole purpose of this thing is to show that if you have a private object, which implements a public interface, then the private object can both query, so this is important, it can, both query the internal state of the object that created it and give you answers. So, it can answer things about the object that it comes from. But it can also maintain auxiliary information to control how long this object, this query object is available to you.

(Refer Slide Time: 13:32)



IIT Madras
BSc Degree

Summary

- Can provide controlled access to an object
- Combine private classes with interfaces
- External interaction is through an object of the private class
- Capabilities of this object are known through a public interface
- Object can maintain instance variables to track the state of the interaction



Madhavan Mukund

Controlled interaction with objects

Programming Concepts using Java

6/6

So, to summarize, through a combination of private classes and interfaces, you can provide this very controlled access to an object. So, you basically ship out from the class that you want to provide this access to. So, you are inside the railway booking class and you want others to be able to answer questions about this thing, ask questions about your database, so you ship out an object which can answer these questions. And this object that you ship out what is to be done with it, is specified through public interface. So, the user knows what this object can do, because it implements a public interface.

At the same time, that object can maintain inside it some instance variables, some local state, to control how this user behaves. So, whether it is valid at all to begin with or if there is a limited validity and so on, so you can keep track of the state of the interaction, and at some point, you can modify the interaction based on that state. So, all these things become possible once you combine this notion of private inner classes with the notion of a public interface. So, it is a very powerful combination and allows you to do a lot of things and we will see a couple of more examples of this type.